



PDF Download
3658644.3670316.pdf
10 March 2026
Total Citations: 23
Total Downloads: 4179

Latest updates: <https://dl.acm.org/doi/10.1145/3658644.3670316>

RESEARCH-ARTICLE

Zero-Knowledge Proofs of Training for Deep Neural Networks

KASRA ABBASZADEH, University of Maryland, College Park, College Park, MD, United States

CHRISTODOULOS PAPPAS, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

JONATHAN KATZ, University of Maryland, College Park, College Park, MD, United States

DIMITRIOS PAPADOPOULOS, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

Open Access Support provided by:

University of Maryland, College Park

Hong Kong University of Science and Technology

Published: 09 December 2024

Citation in BibTeX format

CCS '24: ACM SIGSAC Conference on
Computer and Communications Security
October 14 - 18, 2024
UT, Salt Lake City, USA

Conference Sponsors:
SIGSAC

Zero-Knowledge Proofs of Training for Deep Neural Networks

Kasra Abbaszadeh
University of Maryland
College Park, MD, USA
kasraz@umd.edu

Jonathan Katz
Google
Washington, DC, USA
University of Maryland
College Park, MD, USA
jkatz2@gmail.com

Christodoulos Pappas
Hong Kong University of Science and Technology
Hong Kong, China
cpappas@connect.ust.hk

Dimitrios Papadopoulos
Hong Kong University of Science and Technology
Hong Kong, China
dipapado@cse.ust.hk

Abstract

A zero-knowledge proof of training (zkPoT) enables a party to prove that they have correctly trained a committed model based on a committed dataset without revealing any additional information about the model or the dataset. An ideal zkPoT should offer provable security and privacy guarantees, succinct proof size and verifier runtime, and practical prover efficiency. In this work, we present KAIZEN, a zkPoT targeted for deep neural networks (DNNs) that achieves all these goals at once. Our construction enables a prover to iteratively train their model via (mini-batch) gradient descent, where the number of iterations need not be fixed in advance; at the end of each iteration, the prover generates a commitment to the trained model parameters attached with a succinct zkPoT, attesting to the correctness of the executed iterations. The proof size and verifier time are independent of the number of iterations.

Our construction relies on two building blocks. First, we propose an optimized GKR-style (sumcheck-based) proof system for the gradient-descent algorithm with concretely efficient prover cost; this allows the prover to generate a proof for each iteration. We then show how to recursively compose these proofs across multiple iterations to attain succinctness. As of independent interest, we propose a generic framework for efficient recursive composition of GKR-style proofs, along with aggregatable polynomial commitments.

Benchmarks indicate that KAIZEN can handle the training of complex models such as VGG-11 with 10 million parameters and batch size 16. The prover runtime is 15 minutes per iteration, which is 24× faster than generic recursive proofs, with prover memory overhead 27× lower. The proof size is 1.63 megabytes, and the verifier runtime is only 130 milliseconds, where both are independent of the number of iterations and the size of the dataset.

CCS Concepts

• Security and privacy → Cryptography.



This work is licensed under a Creative Commons Attribution International 4.0 License.

CCS '24, October 14–18, 2024, Salt Lake City, UT, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0636-3/24/10
<https://doi.org/10.1145/3658644.3670316>

Keywords

zero-knowledge proofs; machine learning; deep neural networks; proof of training; incrementally verifiable computation

ACM Reference Format:

Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. 2024. Zero-Knowledge Proofs of Training for Deep Neural Networks. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*, October 14–18, 2024, Salt Lake City, UT, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3658644.3670316>

1 Introduction

Machine learning with deep neural networks (DNNs) has received unprecedented attention in recent years. At the same time, this widespread use of neural networks has raised concerns about the provenance and integrity of DNN models; see, e.g., a recent blog post [2] from Google about integrity issues in the context of general machine learning models. Such concerns can arise at any stage of the model development process. In this work, we focus on the problems of *proofs of training* (PoTs) for ensuring the integrity of the training stage. Here, roughly speaking, a model owner wants to convince a verifier that a model $\mathcal{M}$ was correctly trained—i.e., trained using a publicly known learning algorithm and public specifications (e.g., the model architecture and batch size)—using a specific dataset $\mathcal{D}$.¹ When the model and/or dataset are committed (i.e., not publicly available), one can also consider *zero-knowledge* PoTs (zkPoTs) that reveal no additional information about $\mathcal{M}$ or $\mathcal{D}$ beyond the correctness of the training procedure. At the most basic level, a (zk)PoT can be used by a model owner to substantiate the provenance of a model they release; this can be useful even when the model or the dataset is public since the cost to train a model can be orders of magnitude more expensive than verifying that training was correct. A (zk)PoT can also be useful for other applications:

- **Arbitrating copyright claims.** There is growing concern about models being trained on copyrighted data [39], and a model owner may want to prove that their model was trained *without* using some particular copyrighted data item. To achieve this goal, the model owner can commit to the dataset $\mathcal{D}$ used to train the model, prove using known techniques [10, 52] that the data item

¹We note this is different from *proofs of inference* [32, 48, 50, 61] where, given a model $\mathcal{M}$ and an input x , the goal is to prove that the classification of x was done using $\mathcal{M}$.

in question is not present in $\mathcal{D}$, and then use a (zk)PoT to show that the model was trained using the committed dataset $\mathcal{D}$.

- **Distributed training.** A model owner might distribute the task of training a model across multiple untrusted workers [49]. In this setting, the model owner may send portions of the dataset to each worker and have them return partial results, which can then be aggregated into a final model. PoTs can allow workers to prove that the partial model they return was trained correctly.
- **Proof of ownership.** A deep learning model $\mathcal{M}$ might be stolen or extracted using model-inference attacks [58]. In that case, the real model owner can prove that they were the ones who trained $\mathcal{M}$ by committing to the dataset $\mathcal{D}$ they used to train the model and then using a PoT to prove that $\mathcal{M}$ was trained using $\mathcal{D}$. An adversary would not be able to generate such a proof without access to $\mathcal{D}$; even with such access, they would have to invest the resources necessary to train the model from scratch.

An ideal zkPoT would have strong security guarantees, be succinct², and not impose too much overhead on the prover. As we discuss next, however, known techniques fall short of achieving at least one of these properties when it comes to proofs of training for DNNs.

1.1 Prior Work and Limitations

Jia et al. [40] proposed a proof of training for DNNs. Unfortunately, their construction does not offer cryptographically strong security guarantees, and recent work [30, 31] has identified various concrete attacks on their scheme. Moreover, their construction is neither zero-knowledge nor succinct; it reveals model weights and data items to the verifier who partially re-executes training. Recently, Garg et al. [34] constructed zkPoTs with provable security; however, their construction is not succinct and, as a result, only supports basic learning algorithms such as logistic regression but cannot support DNNs due to the significant verification overhead.

In principle, one can build a zkPoT using generic zero-knowledge proofs. Unfortunately, despite recent advances in such constructions [4, 25, 38, 62], they are still not sufficiently scalable to practically instantiate zkPoTs for complex models like DNNs. In particular, succinct zero-knowledge proofs (zkSNARKs) [12, 42, 53] incur prohibitive prover costs in terms of both the proof-generation time (at least 1000× slowdown compared to the training time) and the required memory overhead. On the other hand, constructions of proofs with efficient provers [11, 35, 64] are not succinct.

As already noted, existing works on zero-knowledge proofs of inference [5, 48, 50, 61] solve a different (and somewhat easier) problem than the one we are considering here. While some of the optimizations they propose are also useful in our setting, the techniques used in those works are not sufficient to yield efficient zkPoTs for DNNs—note, in particular, that DNN training involves multiple iterations, where each iteration can be even 100× more expensive than DNN inference. Eisenhofer et al. [28] used generic SNARKs to address the problem of verifiable machine unlearning, where the goal is to retrain a model by removing particular items from the dataset. The computational complexity of unlearning is fairly low; thus, generic proofs can efficiently support this problem.

1.2 Our Contributions

In this work, we propose a technique for constructing efficient and succinct zkPoTs for DNN models that are trained using multiple iterations of the mini-batch gradient-descent algorithm. We also implement and evaluate KAIZEN,³ an instantiation of our approach. Conceptually, our work involves two components: an optimized proof system for gradient-descent iterations, and a framework for efficient recursive composition of proofs across multiple iterations.

Efficient proofs of gradient descent. First, we construct an optimized *proof of gradient descent* (PoGD), a scheme for proving the correctness of a single gradient-descent iteration. Our PoGD is constructed from GKR-style (sumcheck-based) proof systems [62, 65, 66] since they offer concretely efficient prover overhead and succinct verification. Moreover, such proofs can be optimized for matrix operations such as matrix multiplication or convolution used in gradient-descent for DNNs to obtain sublinear proof-generation time [50, 57]; this is in contrast to generic proofs, which have prover overhead at least linear in the complexity of the computation.

Recursive composition of sumcheck-based proofs. Although our PoGDs offer succinct proofs for each gradient-descent iteration, they do not suffice to achieve succinct proofs for DNN training overall because such training involves a number of iterations that are typically linear in the size of the dataset. Thus, having the PoT include a PoGD for each iteration would result in a linear-sized proof. Alternately, using a single GKR-style proof for all iterations would result in a linear prover memory and verification overheads; note such proofs have verification linear in the computation depth.

Instead, we use recursive proof composition, also referred to as *incrementally verifiable computation* (IVC) [9, 13, 59], to achieve succinctness. Roughly, at each iteration we prove both that the iteration was performed correctly and (recursively) that there is a valid proof of correctness for all previous iterations. In this way, the proof generated at the final iteration demonstrates the correctness of the entire computation with a proof size and verification cost independent of the iteration number. Additionally, the incremental nature of an IVC allows for prover memory cost only proportional to the complexity of one iterative step, not the entire computation.

The challenge in recursively composing our sumcheck-based PoGDs is the relatively large circuit size of the verifier algorithm, which imposes a significant recursion cost; this is in contrast to generic IVC schemes [13, 17, 44], which offer efficient recursion cost but incur substantial prover cost overall for the gradient-descent computation since they rely on generic zero-knowledge proofs. To overcome this, we propose a framework for minimizing the recursion overhead when building IVC from sumcheck-based proofs. In particular, inspired by Halo [17] and follow-up works [15], we use aggregation schemes to reduce the cost of verifying the polynomial commitments used in the PoGDs for all executed iterations.

A difficulty here is that sumcheck-based proofs use commitments to *multivariate* polynomials instead of the *univariate* polynomials involved in prior work [15, 17]; existing aggregation schemes for polynomial commitments only handle univariate polynomials, and naively extending them to support multivariate polynomials would result in quasilinear prover overhead. To mitigate this limitation,

²Succinctness means proof size and verifier time are sublinear in the computation size.

³KAIZEN is a Japanese term translated as “change for the better”.

and as a key building block for our recursion framework, we propose an aggregation scheme for multivariate polynomial commitments with linear prover overhead and logarithmic verifier overhead.

We further improve the overhead of recursive composition of sumcheck-based proofs with sumcheck-specific optimizations for lightweight hash functions [6] and aggregation of the evaluations of circuit-wiring predicates; To our knowledge, building IVC from GKR-style and sumcheck-based proofs has not been considered by prior work, and our framework and proposed techniques might be of independent interest as stand-alone primitives.

Implementation and evaluations. We evaluate KAIZEN on several well-known architectures [46, 47, 56], e.g., a convolutional model VGG-11 with 10 million parameters, 11 layers, and batch size 16. The model is trained on the CIFAR-10 dataset. Our prover time is 15 minutes per iteration, and independent of the number of iterations and the dataset size, the proof size is 1.66 megabytes, and the verifier time is only 130 milliseconds. Moreover, we compare KAIZEN with generic IVCs [17, 26, 44] as the baseline, where we achieve $24\times$ faster prover time and at least $27\times$ less prover memory usage.

1.3 Concurrent Work

In concurrent work, Bünz et al. [21] also propose an IVC based on the GKR protocol. They first construct an accumulation scheme [19] for the GKR protocol, and then utilize the ProtoStar compiler [18] to transform the accumulation scheme to an IVC. Their construction has quasilinear prover runtime in the size of the iteration circuit; in contrast, our IVC offers a linear-time prover overhead.

In other concurrent work, Bünz et al. [22] give an aggregation scheme for non-homomorphic hash-based vector commitments based on error-correcting codes, and then show how to compile it to an IVC. The aggregation technique they use is similar to ours, but requires applying the commitment and aggregation schemes to vectors of the same size as the iteration circuit whereas in our work only inputs and outputs of the circuit are committed. Thus, their scheme would incur a significant prover overhead when applied to complex iterated computations such as DNN gradient descent.

1.4 Organization of the Paper

In Section 2, we introduce notation and include some background information on cryptographic primitives, zero-knowledge proofs, and DNNs. In Section 3, we present an overview of our techniques. Section 4 describes our PoGD scheme. We introduce our aggregation scheme in Section 5, and use this as a building block of our recursion framework presented in Section 6. In Section 7, we present KAIZEN, which we evaluate and compare to generic IVCs in Section 8.

2 Preliminaries

We use $\mathbb{F}$ to denote a finite field. We let λ be the security parameter, and negl be a negligible function. We define $[n] := \{0, 1, \dots, n-1\}$. We use bold lowercase letters $\mathbf{x}, \mathbf{y}$ for vectors, and bold uppercase letters $\mathbf{X}, \mathbf{Y}$ for matrices. For a vector $\mathbf{x}$, we use both x_i and $\mathbf{x}[i]$ to denote the i th element of $\mathbf{x}$, and for a matrix $\mathbf{X}$, both $X_{i,j}$ and $\mathbf{X}[i, j]$ indicate the element in row i and column j . We use $\mathbf{X}[i, :]$ to denote the i th row and $\mathbf{X}[:, j]$ to denote the j th column of the matrix $\mathbf{X}$. We use $\circ$ for the Hadamard (element-wise) product.

Merkle tree. A Merkle tree [52] MT is a data structure that can be used to commit to a vector. It consists of the following algorithms:

- **Commit:** on input a vector $\mathbf{x}$ and randomness r , returns the root ρ of the Merkle tree as a commitment to the vector.
- **Open:** on input a vector $\mathbf{x}$, randomness r , and opening index i , returns an element x_i and proof p_i .
- **Verify:** on input a commitment ρ , index i , element x_i , and proof p_i , returns accept or reject.

Commitment time is linear, proofs and verifier times are logarithmic.

Multilinear extensions. Let $V : \{0, 1\}^\ell \rightarrow \mathbb{F}$ be a function. The *multilinear extension* of V is defined as the unique multilinear polynomial $\tilde{V} : \mathbb{F}^\ell \rightarrow \mathbb{F}$ with $\tilde{V}(\mathbf{x}) = V(\mathbf{x})$ for all $\mathbf{x} \in \{0, 1\}^\ell$. For $\mathbf{b} \in \{0, 1\}^\ell$, let $\beta_{\mathbf{b}} : \{0, 1\}^\ell \rightarrow \{0, 1\}$ denote the function with $\beta_{\mathbf{b}}(\mathbf{x}) = 1$ if $\mathbf{b} = \mathbf{x}$ and $\beta_{\mathbf{b}}(\mathbf{x}) = 0$ otherwise, and note that $\tilde{\beta}_{\mathbf{b}}(\mathbf{x}) = \prod_{i=1}^\ell ((1 - x_i)(1 - b_i) + x_i b_i)$. We can then compute $\tilde{V}$ as

$$\tilde{V}(\mathbf{x}) = \sum_{\mathbf{b} \in \{0, 1\}^\ell} \tilde{\beta}_{\mathbf{b}}(\mathbf{x}) \cdot V(\mathbf{b}).$$

We can define multilinear extensions of vectors by viewing a vector $\mathbf{v} = (v_0, v_1, \dots, v_{n-1}) \in \mathbb{F}^n$ as a function $v : \{0, 1\}^{\log n} \rightarrow \mathbb{F}$ such that $\forall i \in [n] : v(i) = v_i$. A similar method is applied to the matrices.

2.1 Proofs, Arguments, and Commitments

An interactive proof for relation R with corresponding language L_R is an interactive protocol between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ on joint input x . The protocol satisfies *completeness* if, in an honest execution when the prover additionally holds a witness w with $(x, w) \in R$, the verifier always accepts. *Soundness* guarantees that a malicious $\mathcal{P}$ cannot convince $\mathcal{V}$ to accept when $x \notin L_R$, except with negligible probability. *Knowledge soundness* is a stronger property requiring that if a malicious prover can cause the verifier to accept with high probability on joint input x , then it is possible to extract a witness w from the prover such that $(x, w) \in R$. Such a protocol is called an *argument* if (knowledge) soundness holds against only computationally-bounded provers. Informally, a proof/argument is *zero-knowledge* if a malicious verifier learns no information from the protocol about w other than $(x, w) \in R$. We say a proof/argument is *succinct* if the runtime of $\mathcal{V}$ and the communication between $\mathcal{P}$ and $\mathcal{V}$ are $\text{poly}(\lambda, |x|, \log |w|)$; see Appendix A.1 for formal definitions.

A proof/argument is *public-coin* if the messages sent by the honest verifier consist simply of random challenges. Under certain conditions, a public-coin protocol can be made non-interactive using the Fiat-Shamir transform [33] in the random-oracle model by replacing the challenge for each round with the output of the random oracle evaluated on the transcript of prior rounds.

Incrementally verifiable computation (IVC). IVC schemes [59] enables succinct verification of iterative computations. Let $\mathcal{F}$ be a function, $\omega_0, \dots, \omega_{i-1}$ be witness auxiliary inputs, and z_0 be an initial input; define $\forall k \in [i] : z_{k+1} = \mathcal{F}(z_k, \omega_k)$. IVC allows a prover to incrementally generate a proof π_i substantiating that there exist $\omega_0, \dots, \omega_{i-1}$ such that z_i was correctly computed from z_0 . More formally, an IVC protocol consists of the following algorithms:

- $\mathcal{G}$: on input security parameter, returns public parameters pp .

- $\mathcal{P}$: on input an iteration counter i , initial input z_0 , last output z_{i-1} , auxiliary input ω_{i-1} , proof π_{i-1} , and public parameters pp , returns the next iteration output z_i and a proof π_i .
- $\mathcal{V}$: on input an iteration counter i , initial input z_0 , last output z_i , proof π_i , and public parameters pp , returns accept or reject.

Each π_i is a zero-knowledge argument for the relation:

$$R_{VC_i} = \left\{ \begin{array}{l} (i, z_0, z_i), (\omega_0, \dots, \omega_{i-1}) : \\ \forall k \in [i] : z_{k+1} = \mathcal{F}(z_k, \omega_k) \end{array} \right\}.$$

A canonical technique for constructing an IVC scheme is recursive composition of a baseline succinct non-interactive proof.

Let $\mathcal{P}_b$ and $\mathcal{V}_b$ be the prover and verifier of the baseline proof, respectively. Let $\mathcal{F}_A$ be the augmented function defined as:

$$(\mathcal{F}(z_i, \omega_i), \mathcal{V}_b(i, z_0, z_i, \pi_i)) \leftarrow \mathcal{F}_A(i+1, z_0, z_i, \omega_i, \pi_i).$$

$\mathcal{P}$, on input $i+1, z_i, \omega_i$, and π_i , outputs z_{i+1} and invokes $\mathcal{P}_b$ to generate a proof π_{i+1} that $\mathcal{F}_A(i+1, z_0, z_i, \omega_i, \pi_i) = (z_{i+1}, 1)$. The incremental design of IVC ensures that prover memory overhead, proof size, and verifier overhead are independent of the number of iterations and only proportional to the complexity of $\mathcal{F}_A$.

Polynomial commitments. A polynomial commitment scheme (PCS) enables a prover $\mathcal{P}$ to generate a commitment to a polynomial and later open the polynomial at some input evaluation point. More formally, a PCS consists of the following procedures:

- KeyGen: on input the security parameter, number of variables ℓ , and degree bound d , returns public parameters pp .
- Commit: on input a polynomial f , randomness r , and public parameters pp , returns a commitment σ to the polynomial.
- Open: on input a point x , a polynomial f , the randomness r , and public parameters pp , returns the evaluation $y = f(x)$ and an evaluation opening proof π .
- Verify: on input a commitment σ , point x , evaluation y , and evaluation opening proof π , returns accept or reject.

A PCS is *hiding* if the commitment reveals no information about the polynomial. A PCS satisfies *evaluation binding* if a prover cannot open a commitment σ to two distinct evaluations at any input point x . Knowledge soundness ensures that a prover generating a correct evaluation proof must know the polynomial underlying the commitment. A zero-knowledge PCS guarantees that evaluation proofs reveal no additional information about f other than the value $f(x)$. Formal definitions are provided in Appendix A.1.

2.2 GKR-Based Zero-Knowledge Arguments

Goldwasser et al. [36] proposed an interactive proof system for layered arithmetic circuits, referred to as the GKR protocol, based on the sumcheck protocol. We review the scheme here.

The sumcheck protocol. Fix an ℓ -variate polynomial $f : \mathbb{F}^\ell \rightarrow \mathbb{F}$ with variable-degree bound d . The sumcheck protocol [51] enables a verifier $\mathcal{V}$ to delegate computation of the sum of f over the binary hypercube, i.e., $H = \sum_{b \in \{0,1\}^\ell} f(b)$, to a prover $\mathcal{P}$. At the end of the protocol, $\mathcal{V}$ needs the ability to learn the evaluation of f at a single (random) point $r \in \mathbb{F}^\ell$. The prover time is $O(d^\ell)$, and the verification time and proof size are $O(d\ell)$. The soundness error is $O(d\ell/|\mathbb{F}|)$. We present the scheme in Protocol 1 of Appendix A.2.

The GKR protocol. Let C be a circuit of depth d over a finite field $\mathbb{F}$, with layer 0 the output layer and layer d the input layer, and where each gate in the i th layer takes two inputs from the $(i+1)$ st layer. Let S_i be the number of gates in the i th layer, and $s_i := \lceil \log S_i \rceil$. Define *wiring predicates* $add_i, mult_i : \{0,1\}^{s_i+2s_{i+1}} \rightarrow \{0,1\}$, where add_i (resp., $mult_i$) takes as input a gate label z for layer i , and gate labels x, y for layer $i+1$, then returns 1 iff gate z is an addition (resp., multiplication) gate with inputs x and y . Fixing some input to the circuit, we define $v_i : \{0,1\}^{s_i} \rightarrow \mathbb{F}$ so that $v_i(b)$ is the output of the b th gate in the i th layer. Note that

$$\begin{aligned} \tilde{v}_i(z) = & \sum_{x,y \in \{0,1\}^{s_{i+1}}} \widetilde{add}_i(z, x, y) \cdot (\tilde{v}_{i+1}(x) + \tilde{v}_{i+1}(y)) + \\ & \widetilde{mult}_i(z, x, y) \cdot \tilde{v}_{i+1}(x) \cdot \tilde{v}_{i+1}(y). \end{aligned} \quad (1)$$

$\mathcal{P}$ can prove the evaluation of C as follows. Assume $\mathcal{V}$ has oracle access to $\tilde{v}_0$ and $\tilde{v}_d$ (i.e., the input and output). $\mathcal{V}$ samples a random challenge r_0 and evaluates $\tilde{v}_0(r_0)$. Then, $\mathcal{P}, \mathcal{V}$ run the sumcheck protocol on Equation 1 for $i = 0$, at the end of which $\mathcal{V}$ asks $\mathcal{P}$ for two evaluations of $\tilde{v}_1$, say at points $r_{1,0}, r_{1,1}$. Say $\mathcal{P}$ claims $y_0 = \tilde{v}_1(r_{1,0})$ and $y_1 = \tilde{v}_1(r_{1,1})$. The verifier then chooses $\gamma_0, \gamma_1 \leftarrow \mathbb{F}$ and the parties run the sumcheck protocol to prove that

$$\begin{aligned} \gamma_0 \cdot y_0 + \gamma_1 \cdot y_1 = & \sum_{x,y \in \{0,1\}^{s_2}} \\ & (\gamma_0 \cdot \widetilde{add}_1(r_{1,0}, x, y) + \gamma_1 \cdot \widetilde{add}_1(r_{1,1}, x, y)) \cdot (\tilde{v}_2(x) + \tilde{v}_2(y)) + \\ & (\gamma_0 \cdot \widetilde{mult}_1(r_{1,0}, x, y) + \gamma_1 \cdot \widetilde{mult}_1(r_{1,1}, x, y)) \cdot \tilde{v}_2(x) \cdot \tilde{v}_2(y). \end{aligned}$$

This reduces the problem to a claim of two evaluations of $\tilde{v}_2$. The parties then continue this procedure for d iterations; at the final iteration $\mathcal{V}$ can check the evaluations of $\tilde{v}_d$ using its oracle access. The formal protocol is provided in Protocol 2 of Appendix A.2. The GKR protocol can be made zero-knowledge by using zero-knowledge sumchecks [16, 62] and zero-knowledge polynomial commitments to provide oracle access to $\tilde{v}_0$ and $\tilde{v}_d$. The protocol can be made non-interactive using the Fiat-Shamir transform.

2.3 Deep Neural Networks and Training

A DNN maps an input feature to a prediction by applying a sequence of transformations across L layers. The ℓ th layer consists of a linear operation parameterized by weights $\mathbf{W}_\ell$ followed by application of a non-linear activation function. A layer is *dense* or *convolutional*. In dense layers, the linear operation returns $T = \mathbf{W} \cdot \mathbf{U}$ given an input $\mathbf{U}$ and weights $\mathbf{W}$, and convolutional layers returns $T = \mathbf{W} * \mathbf{U}$, where if $\mathbf{U}$ is of size $u \times u$ and $\mathbf{W}$ is of size $w \times w$, then T is a $(u - w + 1) \times (u - w + 1)$ matrix such that

$$T[i, j] = \sum_{a,b=0}^{w-1} U[i+a, j+b] \cdot W[a, b].$$

Following the linear operation, an activation function is performed; this is $\text{ReLU}(x) = \max(x, 0)$, $\tanh(x) = (e^{2x} - 1)/(e^{2x} + 1)$, each applied coordinate-wise, or $\text{Softmax}(x_i) = (e^{x_i})/(\sum_j e^{x_j})$. A layer may also include a pooling that slides a *window* over the input and extracts one element from each window deterministically, e.g., it returns the maximum or average value in each of the windows.

Training DNNs using mini-batch gradient-descent. When training a DNN, the architecture, i.e., the number of the layers, the

type of each layer, and an initial set of weights, is fixed in advance, and the goal is to optimize the weights at each layer to minimize a specified loss function $\mathcal{L}$ over the training dataset. The training dataset is a set of data items, where each item consists of a feature and associated labels. The most popular optimization method for DNNs is mini-batch gradient-descent [55], which proceeds in a series of *epochs*. In each epoch, the dataset $\mathcal{D}$ is randomly partitioned into batches $B_1, \dots, B_k$, each containing a specified number N of data items. An epoch consists of k iterations (one per batch). In the i th iteration, let $U_{i,0}$ be the concatenation of the input features of the data items in B_i and let $U_{i,\ell}$ (for $\ell > 0$) be the concatenation of outputs of the ℓ th layer given the current weights. In the i th iteration the current weights $\{W_{i-1,\ell}\}_{\ell=1}^L$ are updated to obtain new weights $\{W_{i,\ell}\}_{\ell=1}^L$ via the following steps:

- (1) *Forward pass*: We feed the inputs $U_{i,0}$ to the model, apply layers sequentially, and for $\ell = 1$ to L , compute $U_{i,\ell}$.
- (2) *Backward pass*: Let H_i be the average loss over the batch, and define the gradients $G_{i,\ell} := \partial H_i / \partial W_{i-1,\ell}$ and $R_{i,\ell} := \partial H_i / \partial U_{i,\ell-1}$. These are computed for $\ell = L$ to 1 as:

$$G_{i,\ell} = R_{i,\ell+1} \cdot \frac{\partial U_{i,\ell}}{\partial W_{i-1,\ell}}, R_{i,\ell} = \frac{\partial U_{i,\ell}}{\partial U_{i,\ell-1}} \cdot R_{i,\ell+1}. \quad (2)$$

$R_{i,L+1}$ is evaluated using the labels of the data items. Moreover, the gradients $\partial U_{i,\ell} / \partial W_{i-1,\ell}$ and $\partial U_{i,\ell} / \partial U_{i,\ell-1}$ are evaluated by applying some linear operations and non-linear activations to the inputs $U_{i,\ell-1}$ and weights $W_{i-1,\ell}$, respectively.

- (3) *Update*: Given a learning rate η_ℓ for each layer, the weights are updated as $W_{i,\ell} = W_{i-1,\ell} - \eta_\ell \cdot \bar{G}_{i,\ell}$, where $\bar{G}_{i,\ell}$ indicates the average of $G_{i,\ell}$ over the data items in the batch.

A high-level illustration is presented in Figure 1, and more details are presented in Algorithm 1 of Appendix A.3.

3 Technical Overview

In this section, we provide an overview of our techniques; low-level technical details are still deferred to subsequent sections.

3.1 Sumcheck Proofs for Gradient Descent

Our PoGD is a GKR-style sumcheck-based proof for one iteration of the mini-batch gradient-descent algorithm (cf. Section 2.3). In particular, given a fixed batch of data points and some model weights, PoGD enables the prover to update the weights by applying a gradient-descent iteration, and then, convince the verifier that the update is executed correctly. For the linear portions of the algorithm, we use optimized sumcheck-based proofs with sublinear proof-generation time. For the non-linear operations, we use bit decompositions along with a generic version of the GKR protocol. We discuss each of these components in more detail next.

Linear operations. Linear operations, such as matrix multiplication or convolution, are the most computationally intensive parts of the gradient-descent algorithm. Fortunately, prior work [50, 57] has shown optimized sumcheck protocols for these operations that have proof-generation time sublinear in the runtime of the computation itself. In particular, Thaler [57] proposed a sumcheck proof for the multiplication of two $n \times n$ matrices with $O(n^2)$ prover time; note the underlying computation takes time $\Theta(n^3)$ if naive matrix

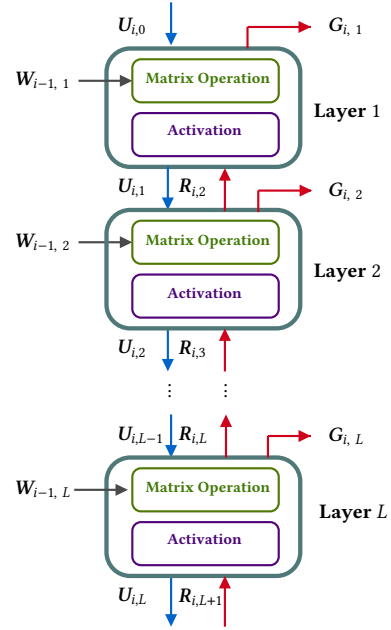


Figure 1: A high-level illustration of the i th gradient-descent iteration. $U_{i-1,0}$ denotes the input features, $U_{i,\ell}$ (for $\ell > 0$) denotes the output of layer ℓ , and $W_{i-1,\ell}$ denotes the current weights of layer ℓ . $R_{i,\ell}$ and $G_{i,\ell}$ are the gradients computed in the backward pass. Blue and red arrows indicate the flow of the forward and backward passes, respectively.

multiplication is used, and even the best matrix multiplication algorithms require time $\omega(n^2)$. More recently, Liu et al. [50] gave a sumcheck protocol for convolution of an $n \times n$ input matrix with a $w \times w$ weights where the prover time is $O(n^2 + w^2)$, asymptotically faster than computing the convolution itself in $O(n^2 \cdot w^2)$ time.

Handling non-linear operations. Sumcheck proofs only support arithmetic operations over finite fields. However, neural networks use non-linear operations that cannot be compactly encoded in an arithmetic circuit, even once the underlying values have been quantized (cf. Section 4.1). We use standard techniques to handle these. For example, proofs about a comparison such as $a \geq 0$ can be handled using bit decomposition: the prover provides the binary representation of a (including a sign bit) as an auxiliary input, and the verification circuit checks that the binary representation is consistent. The sign bit is then taken as the result of the comparison. To reduce the size of the circuit and wiring predicates, we express consistency checks directly as sumcheck instances rather than reducing them to an arithmetic circuit. As another example, we handle exponentiations using piece-wise linear approximations, which have been shown to provide sufficient accuracy [23, 29, 54].

3.2 Recursive Composition of Sumcheck Proofs

A challenge in applying recursive proof composition to our PoGD is the relatively large size of the baseline verification circuit $\mathcal{V}_b$ and hence $\mathcal{F}_A$ (cf. the discussion of IVC in Section 2.1). The circuit $\mathcal{V}_b$ mainly consists of three components: verifying opening proofs

for polynomial commitments, verifying sumcheck messages, and verifying the hashes of messages used to generate the challenge for the Fiat-Shamir transform. We first discuss how we improve the efficiency of each of these components. In particular, we reduce the commitment opening overhead by an aggregatable polynomial commitment scheme, the overhead of verifying sumcheck messages overhead by an evaluation reduction technique, and the Fiat-Shamir overhead by sumcheck-friendly hash functions. Then, we discuss how to maintain the soundness of the IVC scheme for a polynomial number of iterations using the tree-based recursion strategy [14].

Commitment openings. For a GKR-based zkSNARK, the verifier needs to verify openings of several polynomial commitments. It is thus desirable to have a polynomial commitment scheme with a small verification circuit. Known constructions fall short: schemes based on pairings [41] have $O(1)$ verification overhead, but are not compatible with recursive proof composition [9], schemes based on Merkle trees [7, 37, 66] require the verifier to compute $\Omega(\log^2 N)$ hash evaluations for a polynomial of size N . Schemes based on prime-order groups [20, 60] have verification time $\Omega(\sqrt{N})$ making them impractical for recursive proof composition.

Several prior IVC schemes [15, 17] resolved this problem by employing commitment aggregation. Roughly speaking, rather than verifying the opening of commitments in each recursive step, the commitments across all steps are aggregated and the aggregate is verified at the end. Unfortunately, the aggregation scheme used in prior work only applies to polynomial commitment schemes for *univariate* polynomials, whereas for applications to GKR-based proofs we need commitments to *multivariate* polynomials. Naively extending existing univariate schemes to the multivariate case would incur superlinear, i.e., $\Theta(N \log N)$, prover time.

In this work, we propose an aggregation scheme compatible with multivariate polynomial commitments. For polynomials with N coefficients, the scheme has $O(N)$ prover time and $O(\log N)$ proof size and verifier time. Our aggregation technique is inspired by an evaluation reduction technique [36], which is a specific case of aggregation where multiple evaluations of a polynomial are combined into a single evaluation. We generalize this idea to the case of having multiple committed polynomials with multiple evaluation points and also make the scheme zero-knowledge.

Linear-time aggregation schemes for the case of multilinear polynomials have been considered in several prior work for different applications [24, 43]. However, our technique can also be used for multivariate polynomials of higher degree, and can be extended to other cryptographic primitives such as vector commitments.

Sumcheck messages. $\mathcal{V}_b$ must evaluate the multilinear extensions of the wiring predicates at several random points. Evaluating these polynomials requires time linear in the size of the circuit. Interactive implementations of the GKR protocol mitigate this overhead by preprocessing the evaluations [65, 66]. For recursive composition the baseline is non-interactive and preprocessing is not feasible.

We avoid this overhead using an approach similar to aggregation. The various wiring-predicate evaluations are combined into a single evaluation by a sumcheck run. The combined evaluation is passed to the next iteration, and verification is deferred until the end. While a similar effect can be achieved using the aggregation scheme, here, using sumcheck offers concretely better prover performance.

Fiat-Shamir challenges. Due to the Fiat-Shamir transform, $\mathcal{V}_b$ must verify the evaluation of a hash function on many inputs. In particular, $O(d_A \cdot \log |\mathcal{F}_A|)$ hashes need to be verified, where d_A denotes the depth of the augmented function $\mathcal{F}_A$. We use MiMC [3], a sumcheck-friendly hash, to reduce this cost. MiMC applies a low-degree round function for multiple rounds. As observed in prior work [6], we can represent each round of MiMC directly as a sumcheck instance, rather than by explicitly writing it as an arithmetic circuit, and thus further reduce the size of $\mathcal{V}_b$.

Polynomially many iterations. Our commitment aggregation technique (and hence our IVC scheme) only achieves knowledge soundness for $O(1)$ iterations, and there may be attacks when the number of iterations is large [22]. Therefore, in our implementation, we use depth-reduction methods such as tree-based recursion [14] instead of the standard sequential recursion approach. In a tree-based recursion approach, we need to construct a recursion tree whose leaves are the iterations of the actual computation. Each internal node receives proofs for each of its children, applies the verifier circuit to each proof, and aggregates the corresponding polynomial commitments. To achieve constant depth, one can set the arity m of the tree to be polynomially large. In our implementation, the leaves are the gradient-descent iterations and still proved using the PoGD scheme. For the internal nodes the design of the verifier circuit and recursive prover follows the same approach discussed above. Here, the only difference is that instead of validating a single proof, we verify m proofs. Throughout the paper, we describe our protocols based on the standard sequential recursion for ease of exposition. However, our implementation uses tree-based recursion.

3.3 Constructing a zkPoT

We construct a zkPoT for DNNs by recursively composing our sumcheck-based PoGDs across multiple iterations. In doing so, there is one additional complication: we need to ensure that the prover uses a (random) partition of the committed dataset in each epoch. We ensure this as follows. The prover commits to the dataset using a Merkle tree. In each epoch, the prover then partitions the dataset using a public “quasi-random” permutation, e.g., a cubic function, applied to the indices of the n items in the dataset; the key for the permutation is varied for each epoch. During each iteration of an epoch, the prover provides the data items contained in the current partition along with their Merkle proofs. As part of computing gradient-descent iterations, the we additionally check the correctness of the partition as well as the Merkle proofs.

4 Proofs of Gradient Descent

We present our *proof of gradient descent* (PoGD), which is a GKR-style sumcheck-based proof for one mini-batch gradient descent iteration.

4.1 Handling Fixed-Point Operations

Model weights and data items are often represented by real numbers, but we need to encode them as field elements. We fix a field size p large enough to ensure no overflow during the computation. We quantize a real number r as a q -bit integer ($q \ll p$) with f -bit precision as follows: when $0 \leq r \leq 2^{q-f}$, map r to $s = \lfloor r \cdot 2^f \rfloor \in \mathbb{F}_p$, and when $-2^{q-f} \leq r < 0$, map r to $s = p - \lfloor r \cdot 2^f \rfloor \in \mathbb{F}_p$.

Integer arithmetic and comparison. Two numbers can be added by adding their quantizations. For the multiplication of numbers a and b with quantizations s_a and s_b , respectively, we need to scale $z = s_a s_b$ by a factor of 2^{-f} . For this purpose, we ask the prover to provide $z_0, \dots, z_{q-1}$, the bit-decomposition of z , as an auxiliary input along with a sign bit z_q with $z_q = 1$ indicating negativity; The verifier checks whether the provided bit-decomposition consists of binary values consistent with z , and returns the truncated output; i.e., if $z = (-1)^{z_q} \sum_{i=0}^{q-1} z_i \cdot 2^i$ output $(-1)^{z_q} \cdot \sum_{i=0}^{q-f-1} z_{i+f} \cdot 2^i$.

A similar technique can be used for comparisons. Say we need to check whether $z \geq 0$. The prover can provide the bit-decomposition of z , including a sign bit; this can be easily checked by the verifier, and the result of the comparison is then the sign bit. We can also handle divisions. To compute the quotient a/b , where s_a and s_b are the quantizations of a and b , respectively, the prover provides a quotient z and a remainder r . The verifier then checks whether $s_a = z \cdot s_b + r$ and $r < s_b$ and returns $z \cdot 2^f$ as the output. To further reduce the circuit size, we accumulate scaling factors for each layer and defer scalings to the output of the layer of the DNN model.

Sumcheck for binary operations. Scalings and bit-decomposition consistency checks can be handled by a field arithmetic circuit and verified by applying a generic proof to the circuit. However, for complex models, such binary operations blows up the size of the circuit and wiring predicates. To overcome this, we express these operations directly as sumcheck instances. In particular, let z be an n -sized vector of values that the prover is required to provide their bit-decomposition. Let Z_{Bit} be a matrix of size $n \times q$ and z_{Sign} be an n -sized vector, where $Z_{\text{Bit}}[i, \cdot]$ is claimed to be the bit-decomposition of $z[i]$, and $z_{\text{Sign}}[i]$ indicates the associated sign bit. To check whether Z_{Bit} and z_{Sign} are binary, the verifier sends challenges α, r_x, r_y , and parties run the sumcheck protocol on:

$$0 = \sum_{x \in \{0,1\}^{\log n}, y \in \{0,1\}^{\log q}} \tilde{\beta}_x(r_x) \cdot \tilde{\beta}_y(r_y) \cdot (\alpha \cdot \tilde{z}_{\text{Sign}}(x) \cdot (1 - \tilde{z}_{\text{Sign}}(x)) + \tilde{Z}_{\text{Bit}}(x, y) \cdot (1 - \tilde{Z}_{\text{Bit}}(x, y))). \quad (3)$$

The verifier then sends another challenge r_z , and parties verify the consistency of the bit-decompositions by running a sumcheck on:

$$\tilde{z}(r_z) = \sum_{x \in \{0,1\}^{\log n}, y \in \{0,1\}^{\log q}} \tilde{\beta}_x(r_x) \cdot \tilde{Z}_{\text{Bit}}(x, y) \cdot (1 - 2 \cdot \tilde{z}_{\text{Sign}}(x)) \cdot 2^y. \quad (4)$$

We can handle scalings similarly. In particular, we only need to restrict the range of x and y to the proper intervals in Equation 4; the range of x selects which values are targeted and must be scaled, and the range of y determines what scaling factor we apply.

Exponentiation. Natural exponentiation is used in functions such as Softmax and tanh. In practice, weights and data items are typically normalized to the interval $[-1, 1]$. Similar to prior work [34, 54], we employ the following piece-wise approximation for e^x :

$$e^x \approx \begin{cases} 0 & x < -1/2 \\ \frac{1}{2} + x & -1/2 \leq x \leq 1/2 \\ 1 & x > 1/2 \end{cases}$$

4.2 Our PoGD Design

The prover $\mathcal{P}$ updates weights W_{i-1} to W_i using a data batch B_{i-1} (cf. the discussion of gradient-descent in Section 2.3); the PoGD uses sumcheck proof messages to convince a verifier $\mathcal{V}$ that the update is done correctly. For this purpose, $\mathcal{P}$ uses a polynomial commitment scheme to give $\mathcal{V}$ oracle access to the inputs and outputs of the algorithm (i.e., $\tilde{W}_i, \tilde{W}_{i-1}, \tilde{B}_{i-1}$), and also auxiliary inputs denoted by $\tilde{\text{AUX}}_i$, including bit-decompositions, quotients, and remainders; note that W_i, W_{i-1} , and AUX_i are concatenations of $W_{i,\ell}, W_{i-1,\ell}$, and $\text{AUX}_{i,\ell}$, respectively for all layers $\ell = 1$ to L .

PoGD then proceeds in several phases. In the first phase, parties verify the update step of the algorithm, the second phase verifies the backward pass, and the third phase verifies the forward pass. Once sumcheck messages are generated, parties combine random evaluations of inputs and outputs received from different phases together by running additional sumcheck instances. $\mathcal{V}$ verifies the final evaluations using its oracle access. For linear operations, parties run sublinear sumcheck protocols [50, 57], and for non-linear operation they run a generic GKR proof, e.g., Virgo++ [65] in our implementation. Below, we sketch the four phases of our PoGD.

Phase 1: For $\ell = 1$ to L , $\mathcal{V}$ asks for a random evaluation of the trained weights $\tilde{W}_{i,\ell}$. Having this evaluation, parties run GKR on $W_{i,\ell} = W_{i-1,\ell} - \eta \ell \cdot \tilde{G}_{i,\ell}$, where the run ends with random evaluations of the inputs $\tilde{W}_{i-1,\ell}$ and $\tilde{G}_{i,\ell}$. The former proceeds to Phase 4, and the latter proceeds to Phase 2 to bootstrap the next sumchecks.

Phase 2: For $\ell = 1$ to L , having the evaluation of $\tilde{G}_{i,\ell}$ received from Phase 1 and also the evaluation of $\tilde{R}_{i,\ell}$ from the previous iteration (If $\ell \neq 1$) parties verify Equation 2. We use the sublinear sumcheck protocols for linear operations, GKR exploiting bit-decomposition of inputs for non-linear operations, and the sumcheck protocol on Equation 3 and Equation 4 for binary operations of scalings and consistency checks. The run ends with evaluations of $\tilde{W}_{i-1,\ell}, \tilde{U}_{i,\ell-1}, \tilde{R}_{i+1,\ell}$, and $\tilde{\text{AUX}}_{i,\ell}$, where $\tilde{R}_{i+1,\ell}$ proceeds to the next iteration, $\tilde{U}_{i,\ell-1}$ proceeds to Phase 3, and others proceed to Phase 4. For $\ell = L$, parties run GKR on the loss calculation from the actual labels of the data items included in the batch, yielding evaluations of $\tilde{B}_{i-1}$ and $\tilde{U}_{i,L}$ and proceeding to Phase 4 and Phase 3, respectively.

Phase 3: For $\ell = L$ to 1, having evaluations of $\tilde{U}_{i,\ell}$ received from Phase 2 and the previous iteration (if $\ell \neq L$), parties verify the forward pass by running the sublinear sumcheck protocols for the linear operations, GKR for non-linear operations, and the sumcheck protocol for scalings and consistency checks. The run ends with evaluations of inputs of the layer, i.e., $\tilde{W}_{i-1,\ell}, \tilde{U}_{i,\ell-1}$, and $\tilde{\text{AUX}}_{i,\ell}$, where $\tilde{U}_{i,\ell-1}$ proceeds to the next iteration, and $\tilde{W}_{i-1,\ell}$ and $\tilde{\text{AUX}}_{i,\ell}$ proceed to Phase 4; note that $\tilde{U}_{i,0}$ is viewed as an evaluation $\tilde{B}_{i-1}$.

Phase 4: Parties combine all evaluations of $\tilde{W}_{i-1}, \tilde{B}_{i-1}$, and $\tilde{\text{AUX}}_i$ received from the previous phases. In particular, consider a generic ℓ -variate multilinear polynomial f and multiple evaluations of form $\{(x_i, y_i)\}_{i=1}^k$, where $f(x_i) = y_i$. Given challenges $\{\alpha_i\}_{i=1}^k$, we reduce evaluations into a single one by running the sumcheck protocol on:

$$\sum_{i=1}^k \alpha_i \cdot y_i = \sum_{z \in \{0,1\}^\ell} f(z) \cdot \sum_{i=1}^k \alpha_i \cdot \tilde{\beta}_z(x_i). \quad (5)$$

In Phase 4 of our PoGD, parties run this sumcheck by instantiating f with $\tilde{W}_{i-1}$, $\tilde{B}_{i-1}$, and $\tilde{AUX}_i$. At the end of the run, for each of the polynomials, the verifier receives a single evaluation. Along with the evaluation of $\tilde{W}_i$ from Phase 1, the verifier checks these using the oracle access, i.e., by asking for evaluation opening proofs.

We provide more formal details in Protocol 3 and present the security proof for our construction in Appendix B.

Theorem 1. *The PoGD construction presented in Protocol 3 satisfies completeness, knowledge soundness, and zero knowledge.*

Let $s_{in,\ell}$ be the input size and $s_{out,\ell}$ be the output size of the ℓ th layer, N be the batch size, q be the quantization bit-length. We use Orion [63] as our polynomial commitment scheme due to its linear-time and concretely efficient prover and succinct verification. The prover time is $O(\sum_{i=0}^{L-1} N s_{in,\ell} s_{out,\ell} + N q s_{out,\ell})$. The proof size and verifier time are $O(\sum_{i=0}^{L-1} \log^2(N s_{in,\ell} s_{out,\ell}) + \log^2(N q s_{out,\ell}))$, which is dominated by evaluation opening proofs. In the next sections, we discuss how to reduce the verification cost using our aggregation scheme to $O(\sum_{i=0}^{L-1} \log(N s_{in,\ell} s_{out,\ell}) + \log(N q s_{out,\ell}))$ when we compose PoGDs recursively; see Sections 5 and 6. The protocol can be made non-interactive by the Fiat-Shamir heuristic.

5 Aggregatable Commitment Schemes

We present an aggregatable polynomial commitment scheme for *multivariate* polynomials, a key building block for our recursive proof composition framework. In our setting, a prover has generated commitments $\sigma_1, \dots, \sigma_k$ to polynomials $f_1, \dots, f_k : \mathbb{F}^\ell \rightarrow \mathbb{F}$ each is ℓ -variate with variable-degree at most d . The prover then wants to convince a verifier that $y_i = f_i(x_i)$ for some points $x_1, \dots, x_k$ more efficiently than giving k independent evaluation opening proofs. We propose an scheme that allow the prover and verifier to map $\{(\sigma_i, x_i, y_i)\}_{i=1}^k$ to a tuple (σ^*, x^*, y^*) such that σ^* is a commitment to a polynomial f^* of size at most as each input polynomial f_i and such that, with high probability, $f^*(x^*) = y^*$ iff $\forall i : f(x_i) = y_i$; thus a proof that $f^*(x^*) = y^*$ can substantiate the original claim to the verifier. The stronger property of knowledge soundness ensures that a prover who can generate valid aggregation messages and an evaluation opening proof for (σ^*, x^*, y^*) must know f^* and the inputs $\{f_i\}_{i=1}^k$. The scheme is zero-knowledge if the process for generating (σ^*, x^*, y^*) and the eventual opening proof for f^* leak no additional information about the $\{f_i\}_{i=1}^k$. We present more formal definitions of these properties in Appendix C.

5.1 Aggregating Evaluations

We begin by reviewing a technique [36] for aggregating multiple evaluations of a single polynomial. We then show how to generalize that to support distinct polynomials, and to add zero-knowledge.

Single polynomial, multiple points. Consider the case where $f_1 = \dots = f_k = f$ and $\sigma_1 = \dots = \sigma_k = \sigma$, but $\{x_i\}$ are distinct. The prover and verifier interpolate a degree- $(k-1)$ univariate polynomial $L : \mathbb{F} \rightarrow \mathbb{F}^\ell$ with $L(i) = x_i$ for $i \in \{1, \dots, k\} \subseteq \mathbb{F}$. The prover then sends the verifier a polynomial $g = f \circ L : \mathbb{F} \rightarrow \mathbb{F}$ of degree $O(dk\ell)$. The verifier checks that $\forall i : g(i) = y_i$, then, for a random $r \in \mathbb{F}$, the parties set $x^* = L(r)$ and $y^* = g(r)$; in this case, the output polynomial and commitments are $f^* = f$ and $\sigma^* = \sigma$.

We claim that $f(x^*) = y^*$ iff $\forall i : f(x_i) = y_i$ with high probability. Completeness is immediate. Soundness can be argued as follows. Without loss of generality, say $f(x_1) \neq y_1$. If $g = f \circ L$, the verifier rejects since $g(1) \neq y_1$. If $g \neq f \circ L$ then g and $f \circ L$ agree on at most $\deg(g) = O(dk\ell)$ points, and we have $f(x^*) = y^*$ with negligible probability at most $O(dk\ell/|\mathbb{F}|)$.

Multiple polynomials, multiple points. We now extend the above idea to the general case of (possibly) distinct polynomials $\{f_i\}$ and points $\{x_i\}$. As before, the parties define $L : \mathbb{F} \rightarrow \mathbb{F}^\ell$ such that $\forall i : L(i) = x_i$. The prover then sends the k univariate polynomials $g_i = f_i \circ L$, and the verifier checks whether $\forall i : g_i(i) = y_i$. Then, for random challenges $\{\alpha_i\}_{i=1}^k$ and r , the parties set $x^* = L(r)$, $g = \sum_i \alpha_i g_i$, and $y^* = g(r)$. The output polynomial is $f^* = \sum_i \alpha_i f_i$; we defer to Section 5.2 the details of how the parties can generate the commitment σ^* ; we refer to that as *commitment aggregation*.

Again, we claim that $f(x^*) = y^*$ iff $\forall i : f(x_i) = y_i$ with high probability. Completeness holds because

$$f^*(x^*) = (f^* \circ L)(r) = \sum_i \alpha_i (f_i \circ L)(r) = \sum_i \alpha_i g_i(r) = g(r).$$

For soundness, if there exists some $f_i(x_i) \neq y_i$, the prover must send $g_i \neq f_i \circ L$ to pass verification. But then with high probability over choice of $\{\alpha_i\}, r$, we have $y = g(r) \neq (f^* \circ L)(r) = f^*(x^*)$.

Linear-time prover. Naive interpolation of each polynomial g_i would impose prover cost $O(dk\ell \cdot 2^\ell)$ as it requires evaluating a polynomial f_i of size $O(2^\ell)$ at $O(dk\ell)$ points. Fortunately, we can achieve a linear-time prover using a recursive interpolation approach. Define $P_j : \mathbb{F} \rightarrow \mathbb{F}$ of degree $O(dk\ell)$ such that $P_j(i)$ outputs the j th variable of x_i for all $i \in \{1, \dots, k\}$. We observe that

$$g_i = (1 - P_1)(f_i(0, x_2, \dots, x_\ell) \circ L) + P_1(f_i(1, x_2, \dots, x_\ell) \circ L),$$

and so the prover can reduce interpolation of g_i to interpolation of the two polynomials $f_i(0, x_2, \dots, x_\ell) \circ L$ and $f_i(1, x_2, \dots, x_\ell) \circ L$, which can be done in $O(2^{\ell-1})$ time. Then, g_i can be evaluated in $O(dk\ell)$ additional time. This process can be applied recursively to the reduced interpolations to yield a linear runtime (i.e., $O(2^\ell)$) overall overhead for interpolation of each polynomial g_i .

Zero knowledge. Before discussing commitment aggregation, we briefly discuss how to make the above idea zero-knowledge using masking polynomials. For all i , the prover randomly samples an ℓ -variate polynomial h_i and sends a commitment σ'_i to h_i along with an evaluation $v_i = h_i(x_i)$. Given random challenges $\{\beta_i\}$, for all i , the prover then sends $g_i = (f_i + \beta_i h_i) \circ L$, and the verifier checks that $g(i) = y_i + \beta_i v_i$. The rest of the protocol is similar to above; here the output polynomial is $f^* = \sum_i \alpha_i f_i + \alpha_i \beta_i h_i$.

Naively, one could choose each h_i , above, as a random ℓ -variate polynomial of variable-degree at most d . The number of coefficients in each h_i would then be $O(d^\ell)$ and large, making it inefficient in practice to commit to the $\{h_i\}$. In Appendix C, we show it suffices to sample each h_i as $h_i = \sum_{j=1}^\ell h_{i,j}$, where each $h_{i,j}$ is a random *univariate* polynomial (in the j th variable) of degree $d\ell$. The size of each h_i is then $O(d\ell^2)$, and the cost of zero knowledge is $O(kd\ell^2)$.

5.2 Aggregating Commitments

What remains is to show how parties can compute σ^* , a commitment to $f^* = \sum_i \alpha_i f_i$ given $\{\sigma_i\}$, commitments to the $\{f_i\}$; the procedure naturally extends to the case of the zero-knowledge scheme, where

$f^* = \sum_i \alpha_i f_i + \alpha_i \beta_i h_i$, and parties also hold $\{\sigma'_i\}$. Of course, if the underlying commitment scheme is homomorphic then commitment aggregation is trivial. However, practical homomorphic commitment schemes [20, 41, 60] are known only from group assumptions, and cannot be used efficiently for recursive proof composition. Indeed, in our application we use Orion [63], a commitment scheme based on Merkle trees that is not homomorphic. We describe how it is possible to commit to the linear combination of input polynomials for that scheme. Although the discussion is specific to Orion, the technique can be extended to other hash-based schemes [7, 66].

Orion. We briefly review Orion. In that scheme, the coefficients of a polynomial are mapped to the coefficients of a square matrix M , say of size $n \times n$. Then, a linear error-correcting code $E : \mathbb{F}^n \rightarrow \mathbb{F}^p$ is applied to M twice. First, we apply the code to each row of M , yielding an $n \times p$ matrix C_1 . Then, we apply the code to each column of C_1 , yielding a $p \times p$ matrix C_2 . The prover computes a Merkle root for each column of C_2 , and then a Merkle root computed over those p roots is returned as a commitment to the polynomial.

The verifier checks the well-formedness of a commitment by sending a random vector $r \in \mathbb{F}^n$ along with $t = \Theta(\lambda)$ random indices in $[p]$. The prover responds with a vector $v = r^T \cdot M$ along with the corresponding t columns of C_1 and the associated Merkle proofs for the encodings of those columns. The verifier (1) evaluates the codeword $w = E(v)$; (2) for each challenge index i , the returned column $C_1[:, i]$ satisfies $w_i = r^T \cdot C_1[:, i]$; and (3) the Merkle proofs are all correct. Evaluation proofs can be generated similarly, but we omit details. Orion achieves zero knowledge and succinctness by randomized encoding and composing the verification procedure described above with a generic succinct zero-knowledge proof [66].

Aggregation for Orion. Assume the verifier holds commitments $\{\sigma_i\}$ to polynomials $\{f_i\}$ known to the prover, and let $M_i, C_{i,1}, C_{i,2}$ be the corresponding matrices that were generated by the prover when committing to f_i . The prover generates a commitment σ^* to $f^* = \sum \alpha_i f_i$ and sends it to the verifier; let M^*, C_1^*, C_2^* be the corresponding matrices generated during this process. For an honest prover $M^* = \sum_i \alpha_i M_i$, $C_1^* = \sum_i \alpha_i C_{i,1}$, and $C_2^* = \sum_i \alpha_i C_{i,2}$.

On the other hand, if $M^* \neq \sum_i \alpha_i M_i$ then, letting δ be the minimum distance of E , the matrices C_2^* and $\sum_i \alpha_i C_{i,2}$ must differ in at least δ^2 entries.⁴ The key observation is that, in addition to checking the well-formedness of σ^* , the verifier can check that the correct linear relationship holds between random entries of C_2^* and the corresponding entries of $\{C_{i,2}\}$; if E has constant relative distance, it suffices to check that the linear combination relationship holds for $\Theta(\lambda)$ randomly opened entries of each C_2^* and $\{C_{i,2}\}$.

We provide more formal details in Protocol 4 and prove the security of our aggregation scheme in Appendix C.

Theorem 2. *The aggregation scheme presented in Protocol 4 satisfies completeness, knowledge soundness, and zero knowledge.*

The prover cost of is dominated by generating the commitment σ^* , which can be done in $O(d^\ell)$, and evaluating the $\{g_i\}$, which can be done in $O(k \cdot d^\ell)$ time. The proof size is dominated by the Merkle proofs, which have size $O(kd\ell)$; this is also the verifier complexity. The scheme can be made non-interactive via the Fiat-Shamir.

⁴This assumes $C_2^*, \{C_{i,2}\}$ were computed correctly, but in fact, it suffices to check that σ^* is well-formed to ensure that each σ_i were well-formed.

6 Recursive Sumcheck Proofs

We present a generic framework for building IVC from GKR-style sumcheck-based proofs. In particular, our framework relies on the recursive proof composition technique when the baseline proof system $(\mathcal{P}_b, \mathcal{V}_b)$ is instantiated by sumcheck-based non-interactive succinct proofs. When the baseline proof is recursively composed, the prover must generate a proof for the executions of the verifier algorithm $\mathcal{V}_b$. We discuss different components of this algorithm and our techniques to improve the efficiency of each components.

Aggregation instead of opening. $\mathcal{V}_b$ verifies some sumcheck messages for the previous iteration of the augmented function $\mathcal{F}_A$. For this purpose, $\mathcal{V}_b$ receives several random evaluations of the inputs and outputs of $\mathcal{F}_A$; more precisely, evaluations of the multilinear extensions of the inputs and outputs. The inputs are committed by a succinct polynomial commitment scheme; such commitments prevent a blow-up in the input size across iterations. The prover then must provide opening proofs for the evaluations required by $\mathcal{V}_b$ to verify sumcheck messages. Outputs of $\mathcal{F}_A$ might be given to $\mathcal{V}_b$ in plain or to be committed. Here, for simplicity, we assume they are given in plain, and they can be evaluated directly.

To reduce the complexity of $\mathcal{V}_b$, we replace opening proofs for committed polynomials with aggregation messages. In particular, for each i th iteration, $\mathcal{V}_b$ receives two commitment/evaluation instances as follows. First, $\mathcal{V}_b$ receives $\text{agg}_{i-1} = (\sigma_{i-1}^*, x_{i-1}^*, y_{i-1}^*)$, which is the aggregate instance returned the previous iteration; this includes a commitment σ_{i-1}^* and evaluation point (x_{i-1}^*, y_{i-1}^*) , and it is the aggregation of all commitments and their evaluations up to the iteration $i - 2$. Second, $\mathcal{V}_b$ receives $(\sigma_{i-1}, x_{i-1}, y_{i-1})$, which is the instance for the inputs of the previous execution of $\mathcal{F}_A$. Here, again σ_{i-1} is a commitment to the inputs of $\mathcal{F}_A$, and (x_{i-1}, y_{i-1}) is a evaluation point. These two instances are aggregated using our polynomial commitment aggregation scheme (cf. Section 5) and the aggregate output $\text{agg}_i = (\sigma_i^*, x_i^*, y_i^*)$ is passed to the next iteration. Following this approach, we defer opening to the final iteration.

As noted earlier, our aggregation scheme for Orion commitments, as described, is insecure if the number of iterations is large [22]. We must reduce the recursive verification depth using tree-based recursion as discussed previously in Section 3.2.

Handling wiring predicates. $\mathcal{V}_b$ also requires several evaluations of the wiring predicates of $\mathcal{F}_A$ at random points to validate sumcheck messages. Hard-coding all these predicates in $\mathcal{V}_b$ blows up its circuit size and, hence, the recursion cost. Instead, we delegate these evaluations to the prover. We then apply an evaluation reduction sumcheck, i.e., the sumcheck protocol on Equation 5 to combine all wiring predicate evaluations into a single one, which is then passed to the next iteration. The final verifier receives and validates the aggregation of wiring predicate evaluation for all iterations.

Sumcheck-friendly hashing. $\mathcal{V}_b$ verifies challenges for sumcheck proofs and aggregation messages as generated by the Fiat-Shamir transform. Furthermore, $\mathcal{V}_b$ need to evaluate multiple hashes to verify Merkle proofs included in aggregation messages. To reduce the circuit size of this component, we instantiate the hash function with MiMC- p/p , which is a sponge hash function, and each round function is $F_i(x) = (x + k_i)^3 \bmod p$ for some round key k_i . Suppose that we have n number of hash evaluations and let d to denote the

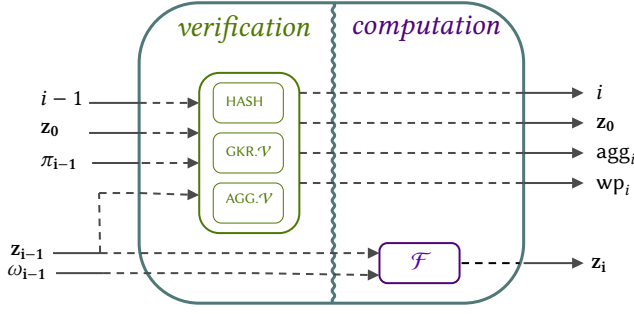


Figure 2: High-level illustration of the augmented function $\mathcal{F}_A$. $\mathcal{F}$ denotes the iteration function. HASH, GKR.V, and AGG.V denote hash evaluation, sumcheck verifier, and aggregation verifier components of $\mathcal{V}_b$, respectively. agg_i denotes the i th aggregate commitment/evaluation instance and wp_i to denote the i th aggregate predicate evaluation.

number of rounds. We can prove each hash round by running a single sumcheck protocol. Let $\mathbf{v}_0$ be the vector of hash outputs, $\mathbf{v}_d$ be the vector of inputs, and $\mathbf{v}_i := F_i(\mathbf{v}_{i-1})$ for all $i \in [d]$. Given an initial challenge r_0 , the prover applies the sumcheck protocol on:

$$\tilde{\mathbf{v}}_0(r_0) = \sum_{z \in \{0,1\}^{\log n}} \tilde{\beta}_z(r_0) \cdot (\tilde{\mathbf{v}}_1(z) + k_1)^3, \quad (6)$$

The run ends with an evaluation $\tilde{\mathbf{v}}_1(r_1)$, which proceeds to the next round. The prover continues this process for all rounds.

Putting everything together. In the i th iteration, $\mathcal{F}_A$ takes as input the iteration counter i , initial inputs z_0 , last output z_{i-1} , an auxiliary input ω_{i-1} , and proof π_{i-1} . The proof includes sumcheck proofs, aggregation messages, aggregate commitment/evaluation instance agg_{i-1} , aggregate wiring predicate evaluation wp_{i-1} , and a commitment and evaluation $(\sigma_{i-1}, x_{i-1}, y_{i-1})$ from iteration $i-1$. $\mathcal{F}_A$ invokes $\mathcal{V}_b(i-1, z_0, z_{i-1}, \pi_{i-1})$, which evaluates required hashes, verifies proof messages, aggregates agg_{i-1} with $(\sigma_{i-1}, x_{i-1}, y_{i-1})$, aggregates wiring predicate evaluations received at the end of the sumcheck verification with wp_{i-1} , and outputs the updated aggregates wp_i and agg_i . Once $\mathcal{V}_b$ is completed successfully, $\mathcal{F}_A$ then evaluates $z_i \leftarrow \mathcal{F}(z_{i-1}, \omega_{i-1})$ and returns $(i, z_0, \text{agg}_i, \text{wp}_i, z_i)$. We illustrate the augmented function in Figure 2. When execution of $\mathcal{F}_A$ is completed, the IVC prover applies hash sumchecks, i.e., sumchecks on Equation 6, to MiMC- p/p hashes and the baseline prover $\mathcal{P}_b$ for other components. Furthermore, the prover needs to generate aggregation messages for the next iteration by running the prover of our aggregation scheme. In the final iteration, the prover additionally requires to generate an evaluation opening proof for the final aggregate commitment/evaluation instance.

In Protocol 5 of Appendix D, we provide more formal details of our recursive sumcheck framework and prove its security.

Theorem 3. *Protocol 4 presents an IVC construction that satisfies completeness, knowledge soundness, and zero knowledge.*

The complexity of the scheme depends on the baseline proof system and the polynomial commitment instantiation. For instance, consider an instantiation, where the GKR-style baseline proof is

implemented by Virgo++ [65] using Orion [63] as the commitment; the prover then is $O(|\mathcal{F}| + |z| + |\omega|)$ per iteration, and proof size and verifier are $O(d \cdot \log |\mathcal{F}| + \log^2(|z| + |\omega|))$, where d denotes the depth of $\mathcal{F}$. These complexities are also preserved when using tree-based recursion to support polynomially many iterations.

7 Zero-Knowledge Proof of Training

We define the notion of the zkPoT primitive, and then we propose KAIZEN, our concrete construction of zkPoT for DNN training.

7.1 Definition

Prior work [34] proposed a monolithic definition of zkPoT, i.e., the prover executes a universal circuit, representing the entire training computation being verified, and generates a proof of the correct execution. Unfortunately, this approach is unsuited for iterative training algorithms as it requires fixing the number of training iterations in advance; the prover should complete the all iterations to generate a verifiable proof. For applications where a model owner incrementally updates their model, it is desirable to have a succinct zkPoT at any stage of the training process, attesting to the correctness of executed iterations. To achieve this goal, we present an incremental definition of the zkPoT primitive, which generates such a succinct zkPoT at the end of each completed iteration.

A consideration of such incremental definitions is ensuring data consistency; each iteration is only given a batch of data items claimed to be randomly sampled from the dataset. To resolve this, assume that there exists some fixed permutation of the dataset for each epoch and the dataset is committed to by a *position-binding* commitment, e.g., its Merkle root. At each iteration, the prover uses the batch consistent with the permutation and provides opening proofs to each data item in the batch. Before running the training iterations, Merkle openings are first validated. More precisely, let C be the iteration circuit in which the permutations are hard-coded. Let $\mathbf{W}_0$ be some initial model and $\mathcal{D}$ be the dataset committed to by ρ . In each i th iteration, C takes as input the iteration counter i , dataset commitment ρ , the last model weights $\mathbf{W}_{i-1}$, a batch of data samples $\mathbf{B}_{i-1} \subseteq \mathcal{D}$, and the batch opening proof $\mathbf{p}_{\mathbf{B}_{i-1}}$. C first validates the opening proof and the consistency of data items in the batch with the hard-coded permutations. If verification passes, it applies the training algorithm and returns weights $\mathbf{W}_i$. In practice, we can compactly hard-code permutations by fixing a key for an efficient pseudorandom permutation for each epoch, which enables C to evaluate the permuted positions required at each iteration.

A zkPoT enables the prover to first commit to the dataset and initial model weights, then sample a batch for each iteration and execute C . Once an iteration is completed, the prover can generate a commitment to the updated weights and a succinct proof that all iterations are correctly executed. More formally, a zkPoT consists:

- **KeyGen:** on input the security parameter, the setup algorithm generates the public parameters pp .
- **DataCom:** on input the training dataset $\mathcal{D}$, randomness $r_{\rho_{\mathcal{D}}}$, and public parameters pp , returns a commitment $\rho_{\mathcal{D}}$.
- **WeiCom:** on input some weights $\mathbf{W}_i$, a randomness $r_{\sigma_{\mathbf{W}_i}}$, and public parameters pp , returns a commitment $\sigma_{\mathbf{W}_i}$.

- **BatchOpen**: on input the dataset $\mathcal{D}$, randomness $r_{\rho_{\mathcal{D}}}$, the iteration counter i , and public parameters pp , returns the batch $\mathbf{B}_{i-1}$ for the i th training iteration with an opening proof $\mathbf{p}_{\mathbf{B}_{i-1}}$.
- **Prove**: on input an iteration counter i , dataset commitment $\rho_{\mathcal{D}}$, initial weights commitment σ_{W_0} , current weights $\mathbf{W}_{i-1}$ with a commitment $\sigma_{W_{i-1}}$, a batch $\mathbf{B}_{i-1}$ with opening $\mathbf{p}_{\mathbf{B}_{i-1}}$, a proof π_{i-1} , and public parameters pp , returns weights $\mathbf{W}_i$ and a proof π_i .
- **Verify**: on input an iteration counter i , the dataset commitment $\rho_{\mathcal{D}}$, the initial weights commitment σ_{W_0} , the trained weights commitment σ_{W_i} , and a proof π_i , returns accept or reject.

A proof-of-training satisfies soundness if the probability of passing verification for an adversary prover, returning inconsistent counter and commitments is negligible. Knowledge soundness ensures that a prover accepted by the verifier must know satisfying weights and the dataset underlying the returned commitments. Zero knowledge guarantees that no additional information about the models and datasets is revealed; see Appendix E.1 for formal definitions. For practicality of the primitive, we require both the proof size and the verifier runtime to be $O(|C|)$ and independent of the number of iterations and the prover cost to be $O(|C| \log |C|)$ per iteration.

7.2 Implementing a zkPoT for DNNs

KAIZEN commits to the dataset by its Merkle root, and the opening of a particular data batch includes Merkle proofs for each of the data items in the batch. Model weights are committed by generating a polynomial commitment to their multilinear extension.

Let PRP be a pseudorandom permutation, e.g., a cubic function, with an initial permutation key s_0 . The key can be updated for each e th epoch to $s_e = \mathcal{H}(e + s_0)$, given a hash function $\mathcal{H}$; this ensures that each epoch trains the model on a randomly sampled partition. Let N be the batch size. For each i th iteration of e th epoch, the training iteration circuit C can be executed as follows:

- (1) $s_e \leftarrow \mathcal{H}(e + s_0)$, $\forall k \in [N] : v_k \leftarrow \text{PRP}(s_e, (i-1)n + k)$.
- (2) Validate $\forall k \in [N] : \text{MT.Verify}(v_k, \rho_{\mathcal{D}}, \mathbf{B}_{i-1}[k], \mathbf{p}_{\mathbf{B}_{i-1}}[k]) = 1$.
- (3) If the verification passes, return $\mathbf{W}_i = \text{GD}(\mathbf{W}_{i-1}, \mathbf{B}_{i-1})$, where GD denotes the the gradient descent algorithm (cf. Algorithm 1).

Consider an instantiation of our recursive sumcheck framework, as outlined in Protocol 5, where the function $\mathcal{F}$ is instantiated by C defined above. The baseline proof system is instantiated by the construction that verifies the execution of C by generating sumcheck messages for step 1 by a generic GKR-style proof [65], step 2 by running sumcheck for MiMC- p/p hash (cf. Section 6), and step 3 via running our PoGD scheme as presented in Protocol 3.

A technical consideration is that in protocol 5, $\mathcal{V}_b$ receives the initial inputs and the last iteration output in plain. In KAIZEN, these include model weights and can be significantly large. To prevent a blow-up in the size of $\mathcal{V}_b$, KAIZEN instead passes commitments to weights. The prover provides the evaluations of weights required for verifying sumcheck messages, and these commitments and their corresponding evaluations are aggregated across the iterations.

We present more formal details of the KAIZEN construction in Protocol 6 and a proof of its security in Appendix E.2.

Theorem 4. KAIZEN is a zero-knowledge proof of training and satisfies completeness, knowledge soundness, and zero knowledge.

Followed by the discussion in Section 4.2, let $s_{\text{in},l}$ be the input and $s_{\text{out},l}$ the output sizes of the l th layer, N the batch size, and q the bit-length of quantization. The KAIZEN prover cost per iteration is $O(\sum_{i=0}^{L-1} N s_{\text{in},l} s_{\text{out},l} + N q s_{\text{out},l})$ both for generating PoGD messages and the recursion overhead. The final proof size and verifier cost are $O(\sum_{i=0}^{L-1} \log^2(N s_{\text{in},l} s_{\text{out},l}) + \log^2(N q s_{\text{out},l}))$, independent of the number of iterations, and hence, the size of the dataset. Although our protocol is described based on the standard sequential recursion, as noted earlier, we use tree-based recursion in our implementation to support any polynomial number of iterations. Here, the asymptotic complexities of the prover and the verifier are preserved.

8 Implementation and Evaluation

We implemented the tree-based variant of KAIZEN and compared it to other approaches. We base the sumcheck and GKR protocols on existing open-source libraries [50, 65]. Our implementation does not support zero-knowledge variants of the sumcheck protocol, the Orion commitment scheme, and aggregation. However, since it only suffices to apply the zero-knowledge extensions to the final iteration, the amortized cost of adding zero knowledge to our implementation would be insignificant. Our code is publicly available at <https://github.com/zkPoTs/kaizen>. Our current implementation could be improved using parallelization, which we did not attempt. We ran all experiments on a Linux VM with 8 physical CPUs (2.80 GHz Intel(R) Xeon(R) Platinum 8370) and 512GB of RAM, and report the average running time of 10 executions.

Protocol instantiation. Our construction works in any finite field. We choose the extension field $\mathbb{F}_{p^2}$ for the prime $p = 2^{61} - 1$. We use MiMC- p^2/p^2 [3] with 80 rounds and SHA-256 as our hash. The pseudorandom permutation PRP for shuffling the dataset is replaced by a cubic function over a field of the size of the dataset. We set $q = 64$ and $F = 32$ for the quantization scheme to ensure no overflow. We use a recursion tree with arity $m = 12$ and depth $d = 4$; hence, our implementation can support up to $12^4 = 20736$ iterations. The generic GKR-style proof is instantiated by the Virgo++ [65]. Our parameters are chosen to achieve $\lambda = 100$ -bit security level.

We instantiated our polynomial commitments with Orion [63]. A difficulty with the Orion protocol is the small relative distance of its linear error-correcting code, which yields a significant number of Merkle tree openings for the aggregation scheme. To reduce this number, instead of parsing the coefficients of polynomials as square matrices, we parse them as matrices with a small constant number of columns and linear-sized rows. Although this yields a quasi-linear evaluation opening time, the commitment and aggregation times remain linear. As in KAIZEN, the evaluation opening is invoked only once at the final iteration. Hence, the effect of this modification on the prover per iteration cost is negligible. We set the column size to one to minimize the aggregation cost, yielding 1813 openings.

Furthermore, we replace MiMC- p^2/p^2 with SHA-256 for up to 4 bottom layers of Merkle trees used in the Orion implementation to reduce the commitment-generation time for large polynomials.

Gradient descent API. We allow a user can select specifications such as the model architectures, the batch size, and learning rate. Then, it outputs sumcheck instances and GKR circuits, required for the gradient-descent iterations. One can also add any number

of convolutional or dense layers to the architecture and specify input, output, and weight sizes. KAIZEN can support a wide range of activations such as ReLU, tanh, Sigmoid, and Softmax. For pooling methods, we can support both average and max pooling.

Furthermore, our implementation supports an arbitrary number of input/output channels for the convolutional layers. In particular, when a convolutional layer has multiple channels, one convolution is required per pair of input/output channels. Multiple convolutions are then turned into multiple FFTs of the same size. The linearity of FFT allows us to reduced all these FFTs into a single FFT by a random linear combination. We should note that a similar technique for batch convolution has been utilized by Balbás et al. [5].

DNN Models and training datasets. We measure the performance of our constructions on three well-known model architectures, LeNet [47], AlexNet [46], and VGG-11 [56]. For AlexNet and VGG-11, we truncated the number of parameters by reducing the number of channels and the size of the output size of the layers. LeNet has 3 convolutional layers, 2 average poolings, 2 dense layers, and a total number of 61,706 trainable parameters. AlexNet has 5 convolutional layers, 3 average poolings, 2 dense layers, and a total number of 4.2 million parameters. Finally, the VGG-11 has 8 convolutional layers, 5 average pooling, 3 dense layers, and a total number of 10.1 million parameters. We use Softmax as the activation for the output layers and ReLU for others. We use the MNIST dataset [27], including 60,000 gray-scale 32×32 images for LeNet, and CIFAR-10 [45], including 60,000 RGB 64×64 images for AlexNet and VGG-11.

8.1 Performance of Our Constructions

We report the overhead of PoGD and KAIZEN provers and verifiers.

Prover cost of PoGD sumchecks. We report the prover time for sumchecks used in PoGD with a breakdown into the four phases of update step, backward pass, forward pass, and evaluation reduction. The batch size is ranged from $N = 4$ to 16. The results are reported and summarized in Table 1; note that the prover cost of polynomial commitments are excluded, and we discuss them in the next reports. As shown in the table, it takes only 13.44 seconds to generate sumcheck messages for the gradient-descent of LeNet with $N = 16$. For AlexNet and VGG-11 it is 57.91 and 179.4 seconds, respectively.

KAIZEN Performance. First, the prover generates the Merkle root of the dataset. This takes only 18 seconds for MNIST and 218 seconds for CIFAR-10, where the Merkle tree is built using $\text{MiMC-}p^2/p^2$ hash function. Then, the prover executes the training iterations and computes proofs for the recursion tree nodes. Here, for every leaf node, it runs the PoGD prover. Then, for recursion (internal) nodes, it aggregates the polynomial commitments of its children and proves the correctness of the aggregation and the verification of its children's proofs. Validating the aggregation of m commitments inside a single circuit would yield a significant memory usage. Instead, we generate proofs for the Merkle openings of each commitment separately. Then, the circuit that validates the children's proofs additionally checks the consistency of those proofs of Merkle openings for each of the polynomial commitments.

The total proving time can be computed as the sum of the prover runtimes of all tree nodes. Finally, we divide this by the number of iterations to get the *amortized runtime per iteration*. We further

break this down and report the individual costs for generating commitments, generating aggregation proofs⁵, the overhead of proving the verifier circuit $\mathcal{V}_b$, and the overhead of the PoGD prover. We consider N from 4–16, and summarize the results in Table 2. As shown in the table, KAIZEN offers a prover runtime of 193 seconds per iteration for LeNet, 474 seconds for AlexNet, and 882 seconds for VGG-11 given a batch size $N = 16$.

The maximum observed prover memory usage across all our experiments was 134.4GB for LeNet, 269.1GB for AlexNet, and 466.3GB for VGG-11. Although the PoGD prover for leaf nodes operates on larger circuits than those used for the recursive nodes, this maximum memory usage occurs in the recursion phase because of the additional data that needs to be stored (e.g., the Merkle tree to produce openings for each polynomial committed with Orion).

We also report the proof size and verifier time with breakdown into sumcheck proofs and polynomial commitment openings proofs. The verification cost is independent of the number of iterations and. For batch size $N = 16$, the proof size is 1.021MB for LeNet, 1.255MB for AlexNet, and 1.627MB for VGG-11. The verifier time is only 73ms for LeNet, 86ms for AlexNet, and 130ms for VGG-11.

8.2 Comparison to Generic IVCs

We compare the performance of KAIZEN with the generic IVC constructions Fractal [26], Halo [17], and Nova [44]. These schemes require the computation to be modeled as an arithmetic circuit, specifically, they rely on the *rank-1 constraint system* (R1CS) circuit representation. We made an effort to minimize the size of the R1CS representation for the iteration circuit C when measuring performance of these schemes. In particular, we handled matrix multiplications using Frievalds' algorithm and handled FFTs using the Cooley–Tukey algorithm instead of naive algorithms considered by the existing implementations [1]. Non-linear operations were handled by using bit-decompositions. For $N = 16$, C has 429M gates for LeNet, 1.6B gates for AlexNet, and 5.1B gates for VGG-11.

Comparisons are summarized in Table 3. As shown, for VGG-11, our prover is 23.7× faster than state-of-art generic IVC, which is Nova. Our verifier time is significantly faster than Halo and Nova. Our proof size is relatively large; however, for practical applications, it might be considered negligible. We extrapolated the performance of generic IVCs based their asymptotic complexities and their reported per-gate performance. We also achieve 27× more efficient memory usage for VGG-11 compared to generic IVCs.

We give a more detailed comparison of KAIZEN and Nova in terms of the prover runtime and memory usage for the case of LeNet with batch sizes ranging from $N = 2$ to 64. Results are reported in Figure 3. As shown, our prover is 6.9–16.1× faster, and we expect that we require 30.7–56.2× less memory overhead.

In fairness, we note that some of the optimizations we used in our implementation might also be applicable to the R1CS-based IVCs. For instance, we expect that the sublinear verification of FFTs can be encoded in R1CS, which might reduce the number of constraints by an order of magnitude. However, no R1CS-based implementations have yet applied these optimizations, and further work would need to be done to incorporate them in existing systems.

⁵The cost of generating commitments to aggregate polynomials returned by the aggregation scheme is also considered as part of the aggregation cost.

	LeNet			AlexNet			VGG-11		
	$N = 4$	$N = 8$	$N = 16$	$N = 4$	$N = 8$	$N = 16$	$N = 4$	$N = 8$	$N = 16$
Phase 1: Update	0.058	0.095	0.164	8.069	11.62	17.07	11.81	17.91	28.96
Phase 2: Backward	2.900	3.851	5.920	5.948	8.546	13.13	28.91	33.80	48.75
Phase 3: Forward	1.688	2.482	4.042	13.96	17.08	22.71	27.51	40.70	65.94
Phase 4: Evaluation	1.032	1.816	3.312	4.425	4.499	4.992	15.93	22.89	35.78
Total Prover (s)	5.676	8.244	13.44	32.42	41.72	57.91	84.15	115.3	179.4

Table 1: Prover runtime as a function of the batch size N when generating sumcheck messages in our PoGD.

	LeNet			AlexNet			VGG-11		
	$N = 4$	$N = 8$	$N = 16$	$N = 4$	$N = 8$	$N = 16$	$N = 4$	$N = 8$	$N = 16$
Commitments	70.04	93.46	106.6	94.72	139.7	233.7	140.3	233.5	452.1
Proof of Agg.	9.877	13.15	19.98	30.33	45.45	75.65	45.76	76.44	142.1
Proof of $\mathcal{V}_b$	26.61	26.63	52.76	104.8	104.5	104.7	106.2	106.8	105.6
Proof of C	5.836	8.563	14.08	33.04	42.95	60.39	84.77	116.5	182.2
Total Prover (s)	112.4	141.8	193.4	262.9	332.6	474.4	377.0	533.2	882.0
Sumcheck Proofs	553.4	575.7	608.9	739.1	790.9	819.0	1045	1117	1154
Comm. Openings	366.8	434.2	412.2	388.9	395.9	436.3	424.4	448.3	473.3
Total Proof (KB)	920.2	1009	1021	1128	1187	1255	1469	1565	1627
Sumcheck Verifier	0.042	0.044	0.050	0.056	0.060	0.063	0.084	0.090	0.105
Comm. Verifier	0.020	0.021	0.023	0.021	0.022	0.023	0.022	0.023	0.025
Total Verifier (s)	0.062	0.065	0.073	0.077	0.082	0.086	0.108	0.113	0.130

Table 2: The amortized prover runtime per iteration, proof size, and verifier runtime of KAIZEN with batch size N .

	Prover (s)			Proof Size (KB)			Verifier (s)		
	LeNet	AlexNet	VGG-11	LeNet	AlexNet	VGG-11	LeNet	AlexNet	VGG-11
Fractal [26]	326,568	1,306,397	5,225,712	243	269	291	0.022	0.026	0.029
Halo [17]	50,850	203,399	813,595	4.98	5.21	5.49	3,970	15,882	63,528
Nova [44]	1,868	7,020	20,880	9.80	10.1	10.3	1,677	6,300	18,735
KAIZEN	193	474	882	1021	1255	1627	0.073	0.086	0.130

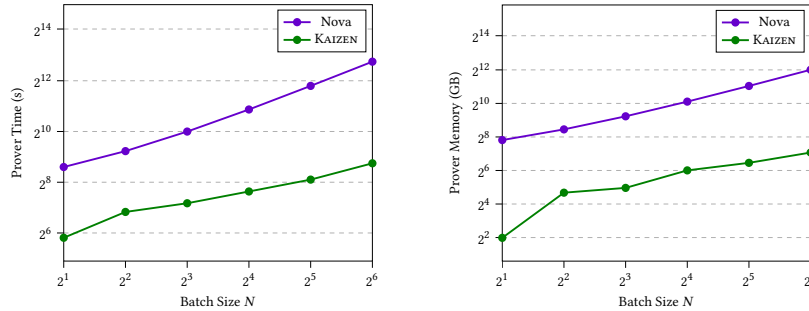
Table 3: The prover runtime per iteration, proof size, and verifier runtime for KAIZEN and generic IVCs given batch size $N = 16$.

Figure 3: The prover time (per iteration) and memory overhead of KAIZEN and Nova for LeNet.

9 Acknowledgments

We would like to thank Srinath Setty and Abhiram Kothapalli for helpful discussions on R1CS size calculations. We also thank Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang for pointing out the knowledge-soundness attack when there are $\Omega(1)$ iterations, and for suggesting tree-based recursion.

Work of Kasra Abbaszadeh and Jonathan Katz was supported by DARPA under contract No. HR00112020025 and by a gift from G0

Labs. Work of Dimitrios Papadopoulos was supported by the Hong Kong RGC under grant 16200721. The views and findings expressed are those of the authors and should not be interpreted as reflecting the position or policy of the Department of Defense or the U.S. Government, and no official endorsement should be inferred.

References

- [1] 2023. Circom Circuits Library for Machine Learning. <https://github.com/socathie/circomlib-ml>.
- [2] 2023. Increasing transparency in AI security. <https://security.googleblog.com/2023/10/increasing-transparency-in-ai-security.html>.
- [3] Martin Albrecht, Lorenzo Grassi, Christian Rechberger, Arnab Roy, and Tyge Tiessen. 2016. MiMC: Efficient Encryption and Cryptographic Hashing with Minimal Multiplicative Complexity. In *Advances in Cryptology—Asiacrypt 2016 (LNCS)*. Springer, 191–219.
- [4] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramanian. 2017. Liger: Lightweight Sublinear Arguments Without a Trusted Setup. In *ACM Conf. Computer and Communications Security 2017*. ACM, 2087–2104.
- [5] David Balbás, Dario Fiore, Maria Isabel González Vasco, Damien Robissout, and Claudio Soriente. 2023. Modular Sumcheck Proofs with Applications to Machine Learning and Image Processing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (<conf-loc>, <city>Copenhagen</city>, <country>Denmark</country>, </conf-loc>)* (CCS '23). Association for Computing Machinery, New York, NY, USA, 1437–1451. <https://doi.org/10.1145/3576915.3623160>
- [6] Alexandre Belling, Azam Soleimani, and Olivier Bégassat. 2023. Recursion over Public-Coin Interactive Proof Systems; Faster Hash Verification. In *ACM Conf. Computer and Communications Security*. ACM.
- [7] Eli Ben-Sasson, Iddo Bentov, Yinon Horeish, and Michael Riabzev. 2018. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In *Intl. Colloquium on Automata, Languages, and Programming (ICALP) (LIPIcs, Vol. 107)*. Schloss Dagstuhl—Leibniz-Zentrum für Informatik, 14:1–14:17.
- [8] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. 2016. Interactive Oracle Proofs. In *Theory of Cryptography (TCC)*. Springer, 31–60.
- [9] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. 2014. Scalable Zero Knowledge via Cycles of Elliptic Curves. In *Advances in Cryptology—Crypto 2014*. Springer, 276–294.
- [10] Josh Benaloh and Michael de Mare. 1994. One-Way Accumulators: A Decentralized Alternative to Digital Signatures. In *Advances in Cryptology—Eurocrypt '93*, Tor Helleseth (Ed.). Springer, 274–285.
- [11] Rishabh Bhadauria, Zhiyong Fang, Carmit Hazay, Muthuramakrishnan Venkitasubramanian, Tiancheng Xie, and Yupeng Zhang. 2020. Liger++: A New Optimized Sublinear IOP. In *ACM Conf. Computer and Communications Security*. ACM, 2025–2038. <https://doi.org/10.1145/3372297.3417893>
- [12] Nir Bitansky, Ran Canetti, Alessandro Chiesa, Shafi Goldwasser, Huijia Lin, Aviad Rubinfeld, and Eran Tromer. 2017. The Hunting of the SNARK. *J. Cryptol.* 30, 4 (oct 2017), 989–1066. <https://doi.org/10.1007/s00145-016-9241-9>
- [13] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. 2013. Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data. In *Symposium on Theory of Computing (STOC)*. ACM, 111–120.
- [14] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. 2013. Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data. In *Symposium on Theory of Computing*. ACM, 111–120.
- [15] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. 2021. Halo Infinite: Proof-Carrying Data from Additive Polynomial Commitments. In *Advances in Cryptology—Crypto 2021, Part I*. Springer, 649–680.
- [16] Jonathan Bootle, Alessandro Chiesa, and Katerina Sotiraki. 2021. Sumcheck Arguments and Their Applications. In *Advances in Cryptology—Crypto 2021, Part I*. Springer, 742–773. https://doi.org/10.1007/978-3-030-84242-0_26
- [17] Sean Bowe, Jack Grigg, and Daira Hopwood. 2019. Recursive Proof Composition without a Trusted Setup. Cryptology ePrint Archive, Paper 2019/1021.
- [18] Benedikt Bünz and Binyi Chen. 2023. Protostar: Generic Efficient Accumulation/Folding for Special-Sound Protocols. In *Advances in Cryptology—ASIACRYPT 2023*, Jian Guo and Ron Steinfeld (Eds.). Springer Nature Singapore, Singapore, 77–110.
- [19] Benedikt Bünz, Alessandro Chiesa, William Lin, Pratyush Mishra, and Nicholas Spooner. 2021. Proof-Carrying Data Without Succinct Arguments. In *Advances in Cryptology—Crypto 2021*. Springer International Publishing, 681–710.
- [20] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. 2018. Bulletproofs: Short Proofs for Confidential Transactions and More. In *IEEE Symposium on Security and Privacy*. IEEE, 315–334.
- [21] Benedikt Bünz and Jessica Chen. 2024. Proofs for Deep Thought: Accumulation for large memories and deterministic computations. Cryptology ePrint Archive, Paper 2024/325. <https://eprint.iacr.org/2024/325> <https://eprint.iacr.org/2024/325>
- [22] Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. 2024. Accumulation without Homomorphism. Cryptology ePrint Archive, Paper 2024/474. <https://eprint.iacr.org/2024/474> <https://eprint.iacr.org/2024/474>
- [23] Gian Carlo Cardarilli, Marco Re, and Luca Di Nunzio. 2021. A pseudo-softmax function for hardware-based high speed image classification. *Sci. Reports* (2021).
- [24] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. 2023. HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates. In *Advances in Cryptology—EUROCRYPT 2023*, Carmit Hazay and Martijn Stam (Eds.). Springer Nature Switzerland, Cham, 499–530.
- [25] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. 2020. Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS. In *Advances in Cryptology—Eurocrypt 2020*. Springer, 738–768.
- [26] Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. 2020. Fractal: Post-quantum and Transparent Recursive Proofs from Holography. In *Advances in Cryptology—Eurocrypt 2020*. Springer, 769–793.
- [27] Li Deng. 2012. The MNIST database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine* 29, 6 (2012), 141–142.
- [28] Thorsten Eisenhofer, Doreen Riepel, Varun Chandrasekaran, Esha Ghosh, Olga Ohrimenko, and Nicolas Papernot. 2023. Verifiable and Provably Secure Machine Unlearning. arXiv:2210.09126
- [29] Alessandro Epasto, Mohammad Mahdian, Vahab Mirrokni, and Manolis Zampetakis. 2020. Optimal Approximation-Smoothness Tradeoffs for Soft-Max Functions. In *Neural Information Proc. Systems (NeurIPS)*. Curran Associates Inc.
- [30] Congyu Fang, Hengrui Jia, Anvith Thudi, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Varun Chandrasekaran, and Nicolas Papernot. 2022. On the Fundamental Limits of Formally (Dis)Proving Robustness in Proof-of-Learning. <https://doi.org/10.48550/ARXIV.2208.03567>
- [31] Congyu Fang, Hengrui Jia, Anvith Thudi, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Varun Chandrasekaran, and Nicolas Papernot. 2023. Proof-of-Learning is Currently More Broken Than You Think. In *European Symposium on Security and Privacy*. IEEE, 797–816.
- [32] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. 2021. ZEN: An Optimizing Compiler for Verifiable, Zero-Knowledge Neural Network Inferences. Cryptology ePrint Archive, Paper 2021/087.
- [33] Amos Fiat and Adi Shamir. 1987. How To Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology—Crypto' 86*, Andrew M. Odlyzko (Ed.). Springer, 186–194.
- [34] Sanjam Garg, Aarushi Goel, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmoody, Guru-Vamsi Policharla, and Mingyuan Wang. 2023. Experimenting with Zero-Knowledge Proofs of Training. <http://ia.cr/2023/1345>
- [35] Irene Giacomelli, Jesper Madsen, and Claudio Orlandi. 2016. ZKBoo: Faster Zero-Knowledge for Boolean Circuits. In *Proceedings of the 25th USENIX Conference on Security Symposium (Austin, TX, USA) (SEC'16)*. USENIX Association, USA, 1069–1083.
- [36] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. 2008. Delegating Computation: Interactive Proofs for Muggles. In *ACM Symposium on Theory of Computing*. ACM, 113–122.
- [37] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. 2021. Brakedown: Linear-Time and Post-Quantum SNARKs for R1CS. Cryptology ePrint Archive, Paper 2021/1043.
- [38] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *Advances in Cryptology—Eurocrypt 2016*. Springer, 305–326.
- [39] Melissa Heikkilä. 2022. This Artist is Dominating AI-Generated Art. And He's not Happy About It. MIT Technology Review.
- [40] Hengrui Jia, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Anvith Thudi, Varun Chandrasekaran, and Nicolas Papernot. 2021. Proof-of-Learning: Definitions and Practice. In *IEEE Symposium on Security and Privacy*. IEEE, 1039–1056.
- [41] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *Advances in Cryptology—Asiacrypt 2010*, Masayuki Abe (Ed.). Springer, 177–194.
- [42] Joe Kilian. 1992. A Note on Efficient Zero-Knowledge Proofs and Arguments. In *Symposium on Theory of Computing (STOC)*. ACM, 723–732.
- [43] Abhiram Kothapalli and Srinath Setty. 2023. HyperNova: Recursive arguments for customizable constraint systems. Cryptology ePrint Archive, Paper 2023/573. <https://eprint.iacr.org/2023/573> <https://eprint.iacr.org/2023/573>
- [44] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. 2022. Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In *Advances in Cryptology—Crypto 2022*. Springer, 359–388.
- [45] Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. [n.d.]. CIFAR-10 (Canadian Institute for Advanced Research). Available at <http://www.cs.toronto.edu/~kriz/cifar.html>.
- [46] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks. In *Adv. in Neural Information Processing Systems*, Vol. 25. Curran Associates, Inc.
- [47] Yan LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324. <https://doi.org/10.1109/5.726791>
- [48] Seunghwan Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2020. vCNN: Verifiable Convolutional Neural Network based on zk-SNARKs.
- [49] Mu Li, David G. Andersen, Jun Woo Park, Alexander J. Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J. Shekita, and Bor-Yiing Su. 2014. Scaling Distributed Machine Learning with the Parameter Server. In *Conference on Operating Systems Design and Implementation*. USENIX Association, 583–598.
- [50] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *Conf. on Computer and Communications Security*. ACM, 2968–2985.

- [51] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. 1992. Algebraic Methods for Interactive Proof Systems. *J. ACM* 39, 4 (oct 1992), 859–868.
- [52] Ralph C. Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology—Crypto '87*. Springer, 369–378.
- [53] Silvio Micali. 2000. Computationally Sound Proofs. *SIAM J. Computing* 30, 4 (2000), 1253–1298.
- [54] Payman Mohassel and Peter Rindal. 2018. ABY3: A Mixed Protocol Framework for Machine Learning. In *Conf. on Computer and Comm. Security*. ACM, 35–52.
- [55] Herbert Robbins and Sutton Monro. 1951. A Stochastic Approximation Method. *The Annals of Mathematical Statistics* 22, 3 (1951), 400 – 407.
- [56] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *Intl. Conf. on Learning Representations, ICLR*.
- [57] Justin Thaler. 2013. Time-Optimal Interactive Proofs for Circuit Evaluation. In *Advances in Cryptology—Crypto 2013*. Springer, 71–89.
- [58] Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. 2016. Stealing Machine Learning Models via Prediction APIs. In *USENIX Security Symposium*. USENIX Association, USA, 601–618.
- [59] Paul Valiant. 2008. Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency. In *Theory of Cryptography (TCC)*. Springer, 1–18.
- [60] R. S. Wahby, I. Tzialla, A. Shelat, J. Thaler, and M. Walfish. 2018. Doubly-Efficient zkSNARKs Without Trusted Setup. In *IEEE Symposium on Security and Privacy*. IEEE, 926–943. <https://doi.org/10.1109/SP.2018.00060>
- [61] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning. In *USENIX Security Symposium*. USENIX Association, 501–518.
- [62] Tiacheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. 2019. Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation. In *Advances in Cryptology—Crypto 2019*. Springer, 733–764.
- [63] Tiancheng Xie, Yupeng Zhang, and Dawn Song. 2022. Orion: Zero Knowledge Proof with Linear Prover Time. In *Adv. in Cryptology—Crypto 2022*. Springer, 299–328.
- [64] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. 2021. QuickSilver: Efficient and Affordable Zero-Knowledge Proofs for Circuits and Polynomials over Any Field. In *Conf. on Computer and Comm. Security*. ACM, 2986–3001.
- [65] Jiaheng Zhang, Tianyi Liu, Weijie Wang, Yinyu Zhang, Dawn Song, Xiang Xie, and Yupeng Zhang. 2021. Doubly Efficient Interactive Proofs for General Arithmetic Circuits with Linear Prover Time. In *Conf. on Computer and Communications Security*. ACM, 159–177.
- [66] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. 2020. Transparent Polynomial Delegation and Its Applications to Zero Knowledge Proof. In *IEEE Symposium on Security and Privacy*. 859–876.